

Technical Implementation Blueprint

Cost- and Degradation-Aware Energy Management of a Grid-Connected
PV-Battery Microgrid — Mathematical Formulation, MATLAB/Simulink
Implementation, Simulation Methodology and Project Execution Plan

Abhi — B.Tech Electrical Engineering, Assam Engineering College

September 2026

Contents

1	Source & Scope Note	4
2	Part 1 — Project Restructuring & Execution Plan	5
2.1	1.1 Phase-by-Phase Breakdown	5
	2.1.1 Phase 1 — Baseline Heuristic Model (Rule-Based EMS)	5
	2.1.2 Phase 2 — Cost Optimization Without Degradation Constraints (Naive LP)	5
	2.1.3 Phase 3 — Degradation-Aware Optimization	6
	2.1.4 Phase 4 — Comparative Benchmarking & Sensitivity Analysis . .	6
	2.1.5 Phase 5 — MPC & Real-Data Validation	7
	2.1.6 Phase 6 — MATLAB App Designer Dashboard	8
2.2	1.2 Dependency Graph	8
3	Part 2 — System Model	10
3.1	2.1 Subsystems	10
3.2	2.2 Variable & Parameter Definitions	10
3.3	2.3 Parameter Table	10
3.4	2.4 Governing Equations	11
4	Part 3 — Discrete-Time Battery Model	13
4.1	3.1 SOC Recursion	13
4.2	3.2 Simultaneous Charge/Discharge — Should You Use a Binary Variable?	13
4.3	3.3 LP-Compatibility	14
5	Part 4 — Power Balance	15
5.1	4.1 Sign Convention	15
5.2	4.2 Complete Balance Equation	15
5.3	4.3 Numerical Example (One Time Step)	15
6	Part 5 — Rule-Based EMS (Phase 1)	16
6.1	5.1 Priority Order	16
6.2	5.2 Pseudocode	16

6.3	5.3 MATLAB Implementation	16
7	Part 6 — Linear Programming EMS (Phase 2)	18
7.1	6.1 Decision Variables	18
7.2	6.2 Objective Function	18
7.3	6.3 Equality Constraints ($A_{eq}x = b_{eq}$)	18
7.4	6.4 Inequality Constraints ($Ax \leq b$) — SOC Bounds	18
7.5	6.5 Bounds ($lb \leq x \leq ub$)	18
7.6	6.6 MATLAB: Building the LP for linprog	19
8	Part 7 — Tariff Model	21
8.1	7.1 Design Requirement	21
8.2	7.2 MATLAB Tariff Construction	21
9	Part 8 — Net vs. EXIM Metering Context	23
9.1	8.1 Electrical Architecture vs. Metering vs. Regulation	23
9.2	8.2 Behind-the-Meter Battery Effect on Metered Flow	23
9.3	8.3 What You Must Verify Before Claiming Anything Regulatory	23
10	Part 9 — Peak Demand Management (Optional Extension)	25
10.19.1	Energy Charge vs. Demand Charge	25
10.29.2	LP-Compatible Peak Formulation	25
10.39.3	Comparison Ladder	25
11	Part 10 — PV Self-Consumption / Utilization Metrics	26
11.110.1	Definitions	26
11.210.2	MATLAB	26
12	Part 11 — Battery Degradation	27
12.111.1	Choosing a Model	27
12.211.2	Degradation Cost Formulation	27
12.311.3	Linearizing for LP/QP Compatibility	28
12.411.4	MATLAB	28
13	Part 12 — Multi-Objective Formulation (Phase 3)	29
13.112.1	Combined Objective	29
13.212.2	Normalization and Weighting	29
13.312.3	General Weighted-Sum Form (if peak/export terms are added)	29
13.412.4	Solving with quadprog	29
14	Part 13 — Scenario Analysis (Phase 4)	31
14.113.1	Scenario Matrix	31
15	Part 14 — Sensitivity Analysis	33
15.114.1	Parameters to Sweep	33
15.214.2	Recommended Plots	33
15.314.3	MATLAB: Automated Sweep	33
16	Part 15 — Model Predictive Control (Phase 5)	34

16.115.1 Formulation	34
16.215.2 Day-Ahead LP vs. Receding-Horizon MPC	34
16.315.3 Pseudocode	34
16.415.4 MATLAB Boilerplate	34
17Part 16 — Forecast Error	36
17.116.1 Error Model	36
17.216.2 Comparison Protocol	36
18Part 17 — MATLAB Project Structure	37
19Part 18 — Data	38
19.118.1 Minimum Required Data	38
19.218.2 Sources	38
19.318.3 Defensible Synthetic Profile Generation	38
20Part 19 — Simulink Architecture	40
20.119.1 Recommended Split	40
20.219.2 ASCII Block Diagram	40
21Part 20 — Validation	42
21.120.1 Checks	42
21.220.2 MATLAB	42
22Part 21 — Expected Results (No Fabricated Numbers)	43
23Part 22 — Report Boilerplate	45
24Part 23 — Figure and Plot Plan	47
25Part 24 — Viva Preparation	48
26Part 25 — What Is Actually Novel?	50
27Part 26 — Implementation Roadmap	51
28Immediate Next Steps	52

1 Source & Scope Note

This document is derived from the project specification **Plan.md** (“Design & Modelling of a Cost-Optimized EMS for a Grid-Tied PV Microgrid”), which is treated throughout as the **primary source of truth** for scope, phase structure, and terminology. Two reconciliation decisions were made explicit rather than silent:

- **Project assumption:** Plan.md’s six-phase structure (Heuristic → Naive LP → Degradation-Aware Optimization → Benchmarking/Sensitivity → MPC → Dashboard) is used as the project backbone. Peak-demand/peak-shaving content is included as an **optional extension** attached to Phase 3, not as a mandatory seventh phase.
- **Optional implementation choice:** Plan.md lists both Convex/Quadratic Programming and metaheuristics (PSO/GA) as candidate solvers for Phase 3. This blueprint recommends **Quadratic Programming (quadprog)** as the primary path because it keeps the project inside the LP→QP solver family already established in Phase 2, and lists PSO/GA as an optional alternative for comparison, not a requirement.

Every numerical result, plot, and table in this document is a **placeholder** ([INSERT SIMULATION RESULT], [INSERT FIGURE], [INSERT VALUE]) unless explicitly marked otherwise. No simulation has been run to produce this document. All MATLAB code is boilerplate meant to be executed by you, not pre-verified output.

2 Part 1 — Project Restructuring & Execution Plan

2.1 1.1 Phase-by-Phase Breakdown

2.1.1 Phase 1 — Baseline Heuristic Model (Rule-Based EMS)

Item	Detail
Objective	Establish a deterministic rule-based dispatch baseline with no optimization
Inputs	$P_{pv}(k)$, $P_{load}(k)$, battery parameters, $SOC(0)$
Outputs	$P_{batt}(k)$, $P_{grid}(k)$, $SOC(k)$ time series
Mathematical model	If-else state logic (Part 5)
MATLAB implementation	rule_based_ems.m
Required data	PV profile, load profile
Expected plots	Power balance vs. time, SOC vs. time
Validation	Power balance residual ≈ 0 ; SOC never exceeds bounds
Gate to next phase	Baseline dispatch runs cleanly for a full 24 h profile with zero constraint violations
Expected difficulty	Low
Approx. effort	3–5 days
Potential problems	Ambiguous priority ordering during simultaneous PV surplus and low SOC; boundary handling at SOC_{min}/SOC_{max}

2.1.2 Phase 2 — Cost Optimization Without Degradation Constraints (Naive LP)

Item	Detail
Objective	Minimize grid energy cost under ToD tariff via LP, ignoring degradation
Inputs	Tariff vector $C_{grid}(k)$, PV/load profiles, battery parameters
Outputs	Optimal $P_{batt}(k)$, $P_{grid}(k)$, $SOC(k)$; post-hoc degradation cost
Mathematical model	LP (Part 6)
MATLAB implementation	build_lp_problem.m, run_lp_ems.m
Required data	Same as Phase 1 + tariff profile
Expected plots	Dispatch vs. time, cost comparison vs. Phase 1, SOC cycling pattern
Validation	linprog exit flag = 1; power balance holds at every step

Item	Detail
Gate to next phase	Naive LP cost savings demonstrated and post-simulation degradation cost quantified (this contrast is the intended narrative motivating Phase 3)
Expected difficulty	Medium
Approx. effort	1-2 weeks
Potential problems	Infeasibility from overly tight SOC/power bounds; simultaneous charge/discharge artifacts (Part 3)

2.1.3 Phase 3 — Degradation-Aware Optimization

Item	Detail
Objective	Balance grid cost against battery degradation cost in one objective
Inputs	Phase 2 inputs + degradation cost parameters (α , replacement cost, DoD-life curve)
Outputs	Degradation-aware optimal dispatch; comparison against Phase 2
Mathematical model	Weighted multi-term objective (Part 11-12)
MATLAB implementation	<code>calculate_degradation.m</code> , <code>degradation-aware build_lp_problem.m</code> variant or <code>quadprog</code> -based solver
Required data	Battery cycle-life vs. DoD curve (literature-derived)
Expected plots	Grid cost vs. degradation cost trade-off curve, SOC cycling comparison (naive vs. aware)
Validation	Degradation-aware schedule shows measurably fewer/shallower cycles than naive schedule for the same input profiles
Gate to next phase	Trade-off behaviour confirmed across α sweep
Expected difficulty	Medium-High
Approx. effort	1.5-2.5 weeks
Potential problems	Non-convexity if degradation term is not linearized/quadratic; weight (α) has no natural units — needs normalization (Part 12)

2.1.4 Phase 4 — Comparative Benchmarking & Sensitivity Analysis

Item	Detail
Objective	Quantify performance across controllers and operating scenarios
Inputs	Outputs of Phases 1–3 across a scenario matrix
Outputs	Comparison table, sensitivity plots
MATLAB implementation	<code>calculate_metrics.m</code> , <code>run_sensitivity_sweep.m</code>
Expected plots	Battery capacity vs. savings, tariff spread vs. utilization, etc. (Part 14)
Validation	Consistent metric definitions applied identically across all controllers
Gate to next phase	Full comparison matrix populated for all three controllers under at least the essential scenarios
Expected difficulty	Medium
Approx. effort	1–1.5 weeks
Potential problems	Combinatorial explosion of scenario × sensitivity runs; keep essential set small (Part 13)

2.1.5 Phase 5 — MPC & Real-Data Validation

Item	Detail
Objective	Rolling-horizon re-optimization with forecast uncertainty; validate on real traces if available
Inputs	Forecast PV/load series with configurable error, real smart-meter/irradiance data if obtainable
Outputs	MPC dispatch trace, comparison vs. day-ahead LP
MATLAB implementation	<code>run_mpc.m</code> , reusing <code>build_lp_problem.m</code>
Expected plots	MPC vs. day-ahead LP dispatch and cost; forecast error sensitivity
Validation	MPC feasibility maintained at every re-optimization step; graceful degradation under forecast error
Gate to next phase	MPC loop runs stably over a full simulated day
Expected difficulty	High
Approx. effort	2 weeks

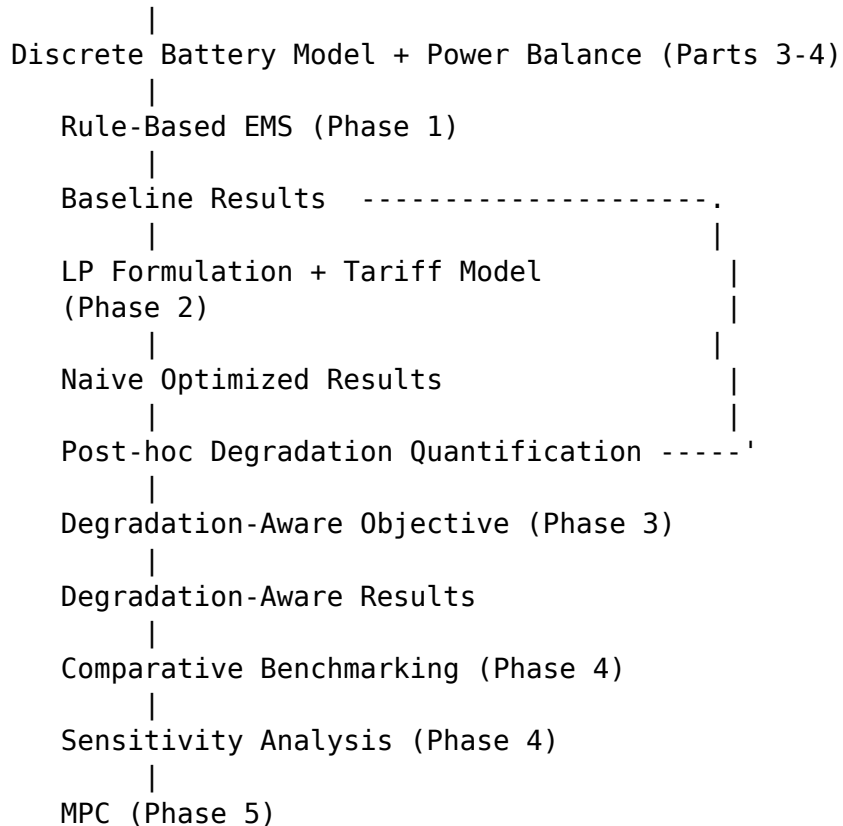
Item	Detail
Potential problems	Real data acquisition may not materialize in time — synthetic fallback must be planned (Part 18)

2.1.6 Phase 6 — MATLAB App Designer Dashboard

Item	Detail
Objective	Interactive GUI for live demonstration
Inputs	All prior simulation outputs, parameterized
Outputs	.mlapp dashboard
Expected difficulty	Medium (UI work, not algorithmic)
Approx. effort	1 week
Potential problems	Time-consuming polish with low marginal grading return if earlier phases are incomplete — this is the correct thing to cut first under time pressure

2.2 1.2 Dependency Graph

System Model (Part 2)



|
Simulink Cross-Check (Part 19, optional)
|
Dashboard (Phase 6)
|
Report + Viva Prep

3 Part 2 — System Model

3.1 2.1 Subsystems

The microgrid consists of four power-exchanging subsystems connected at a common AC bus (or DC bus with a shared inverter, per Part 19): **PV array**, **load**, **battery energy storage system (BESS)**, and **grid connection**.

3.2 2.2 Variable & Parameter Definitions

All time-indexed quantities are sampled at discrete step $k = 1, \dots, N$ with step size ΔT .

Symbol	Meaning	Unit
$P_{pv}(k)$	PV generated power at step k	kW
$P_{load}(k)$	Load demand at step k	kW
$P_{grid}(k)$	Net grid power exchange (signed; see Part 4)	kW
$P_{grid,imp}(k)$	Grid import power (≥ 0)	kW
$P_{grid,exp}(k)$	Grid export power (≥ 0)	kW
$P_{batt}(k)$	Net battery power (signed; see Part 4)	kW
$P_{ch}(k)$	Battery charge power (≥ 0)	kW
$P_{dis}(k)$	Battery discharge power (≥ 0)	kW
$E_{batt}(k)$	Battery stored energy at step k	kWh
$SOC(k)$	State of charge, $E_{batt}(k)/E_B$	fraction (0-1)
ΔT	Simulation time step	h

Project assumption: a 15-minute time step ($\Delta T = 0.25$ h, $N = 96$ per day) is used by default, consistent with the LP horizon specified in the meta-prompt and compatible with ToD tariff blocks that typically change on the hour or half-hour.

3.3 2.3 Parameter Table

Parameter	Symbol	Typical range / note
PV rated power	$P_{pv,rated}$	Project-specific — [INSERT VALUE]
Battery energy capacity	E_B	kWh — [INSERT VALUE]
Battery max charge power	$P_{ch,max}$	kW, often $E_B/2$ for a 0.5C battery
Battery max discharge power	$P_{dis,max}$	kW
Initial SOC	$SOC(0)$	Engineering assumption: 0.5 unless otherwise justified

Parameter	Symbol	Typical range / note
Minimum SOC	SOC_{min}	Engineering assumption: 0.1–0.2 to protect cycle life
Maximum SOC	SOC_{max}	Engineering assumption: 0.9–1.0
Charge efficiency	η_{ch}	Literature-based assumption: 0.95 (Li-ion)
Discharge efficiency	η_{dis}	Literature-based assumption: 0.95
Time step	ΔT	0.25 h (project assumption)
Grid import limit	$P_{grid,imp}^{max}$	Service-connection dependent — [INSERT VALUE]
Grid export limit	$P_{grid,exp}^{max}$	Set by net/EXIM metering agreement — [INSERT VALUE], see Part 8
Battery degradation parameters	$Cost_{replacement}$, DoD-life curve	Literature-derived, Part 11
Tariff parameters	$C_{grid}(k)$, export price	ToD structure, Part 7

3.4 2.4 Governing Equations

Power balance (instantaneous, sign convention formalized in Part 4):

$$P_{pv}(k) + P_{grid,imp}(k) + P_{dis}(k) = P_{load}(k) + P_{grid,exp}(k) + P_{ch}(k)$$

This states that everything generated or imported equals everything consumed, exported, or stored, at every step — the fundamental conservation law the whole EMS must respect.

Battery energy update:

$$E_{batt}(k) = E_{batt}(k-1) + \left(\eta_{ch} P_{ch}(k) - \frac{P_{dis}(k)}{\eta_{dis}} \right) \Delta T$$

Charging power is de-rated by efficiency *before* it adds to stored energy; discharge power is grossed up by the inverse efficiency because more energy must leave the cell than reaches the bus. This is the physical meaning of $\eta_{ch}, \eta_{dis} < 1$: real batteries lose energy to internal resistance on both paths.

SOC:

$$SOC(k) = \frac{E_{batt}(k)}{E_B}, \quad SOC_{min} \leq SOC(k) \leq SOC_{max} \quad \forall k$$

Power limits:

$$0 \leq P_{ch}(k) \leq P_{ch,max}, \quad 0 \leq P_{dis}(k) \leq P_{dis,max}$$

$$0 \leq P_{grid,imp}(k) \leq P_{grid,imp}^{max}, \quad 0 \leq P_{grid,exp}(k) \leq P_{grid,exp}^{max}$$

Inverter/system limit (engineering assumption, if a single shared inverter caps combined PV+battery AC output):

$$P_{pv}(k) + P_{dis}(k) - P_{ch}(k) \leq P_{inv,max}$$

This constraint is only needed if your system architecture uses one shared inverter for PV and battery (a common low-cost hybrid-inverter topology). If PV and battery use separate inverters, omit it — state explicitly in your report which topology you assume.

4 Part 3 — Discrete-Time Battery Model

4.1 3.1 SOC Recursion

Combining the energy update and SOC definition from Part 2:

$$SOC(k+1) = SOC(k) + \frac{\Delta T}{E_B} \left(\eta_{ch} P_{ch}(k) - \frac{P_{dis}(k)}{\eta_{dis}} \right)$$

This is the single equation that couples every time step together in the optimization — it is why the LP in Part 6 has N coupled equality constraints rather than N independent single-step problems.

4.2 3.2 Simultaneous Charge/Discharge — Should You Use a Binary Variable?

If $P_{ch}(k)$ and $P_{dis}(k)$ are both left as independent non-negative continuous variables, nothing in the LP formulation *mathematically* prevents the solver from setting both positive in the same step — which is physically meaningless (you cannot charge and discharge a cell simultaneously) and, worse, lets the solver “burn” energy through round-trip losses to relax a constraint elsewhere in ways that look feasible but are unphysical.

Two approaches:

A. Mixed-integer formulation (rigorous). Introduce a binary $b(k) \in \{0, 1\}$ per step:

$$P_{ch}(k) \leq b(k) P_{ch,max}, \quad P_{dis}(k) \leq (1 - b(k)) P_{dis,max}$$

This guarantees mutual exclusivity but converts the problem from LP to MILP, requiring `intlinprog` instead of `linprog`, with materially higher solve time (relevant once you get to Phase 5’s rolling re-optimization, which must run fast).

B. Unconstrained-but-checked continuous formulation (recommended for this project). Leave $P_{ch}(k), P_{dis}(k) \geq 0$ continuous, and rely on the fact that **because both directions incur round-trip losses ($\eta_{ch}\eta_{dis} < 1$) and the objective minimizes cost**, a cost-minimizing LP solver has no incentive to simultaneously charge and discharge — doing so only wastes energy for no benefit, so the optimal solution will naturally drive at least one of the two to zero at every step in the vast majority of cases. This should then be **validated, not assumed**: after solving, check numerically that $\min(P_{ch}(k), P_{dis}(k)) \approx 0 \ \forall k$ (Part 20 gives validation code). If violations appear (they occasionally can under degenerate/tied-cost conditions), only then escalate to the MILP formulation.

Recommendation: use approach B for Phases 1–4 to keep the project inside LP/QP solvers as the meta-prompt requires, and document the validation check explicitly. Approach A is listed as an **optional implementation choice** if validation reveals persistent violations.

4.3 3.3 LP-Compatibility

Both the SOC recursion and the power limits in Section 3.1–3.2(B) are linear in the decision variables $P_{ch}(k)$, $P_{dis}(k)$, $P_{grid,imp}(k)$, $P_{grid,exp}(k)$, which is what permits the Phase 2 formulation to be solved with `linprog`. This linearity is preserved as long as η_{ch} , η_{dis} are treated as constants (not SOC- or power-dependent) — a standard simplification for a B.Tech-level project, flagged here as a **literature-based assumption**.

5 Part 4 — Power Balance

5.1 4.1 Sign Convention

To keep every equation and MATLAB array unambiguous, this blueprint uses **split non-negative variables** rather than a single signed variable per flow:

- $P_{grid,imp}(k) \geq 0$: power flowing **from** grid **to** microgrid.
- $P_{grid,exp}(k) \geq 0$: power flowing **from** microgrid **to** grid.
- $P_{ch}(k) \geq 0$: power flowing **into** the battery.
- $P_{dis}(k) \geq 0$: power flowing **out of** the battery.

If you prefer signed single variables ($P_{grid}(k) > 0 = \text{import}$, $P_{batt}(k) > 0 = \text{discharge}$), the two are related by $P_{grid}(k) = P_{grid,imp}(k) - P_{grid,exp}(k)$ and $P_{batt}(k) = P_{dis}(k) - P_{ch}(k)$. The split form is recommended for the LP formulation (Part 6) because `linprog` requires linear inequality bounds, and split non-negative variables map directly onto lb/ub without extra sign-logic constraints.

5.2 4.2 Complete Balance Equation

$$P_{pv}(k) + P_{grid,imp}(k) + P_{dis}(k) = P_{load}(k) + P_{grid,exp}(k) + P_{ch}(k) \quad \forall k$$

Rearranged as the equality-constraint form used directly in `linprog`'s `Aeq x = beq`:

$$P_{grid,imp}(k) - P_{grid,exp}(k) + P_{dis}(k) - P_{ch}(k) = P_{load}(k) - P_{pv}(k)$$

5.3 4.3 Numerical Example (One Time Step)

Project assumption (example values only, not simulation results): at step k , suppose $P_{pv}(k) = 4.0$ kW, $P_{load}(k) = 2.5$ kW, and the EMS decides to charge the battery at $P_{ch}(k) = 1.0$ kW, with no discharge and no grid import.

Balance check: $4.0 + 0 + 0 = 2.5 + P_{grid,exp}(k) + 1.0 \Rightarrow P_{grid,exp}(k) = 0.5$ kW.

This example is purely illustrative of how to *use* the equation for a manual sanity check — it is exactly the residual check `validate_power_balance.m` (Part 20) should run automatically over every step of every simulated day.

6 Part 5 — Rule-Based EMS (Phase 1)

6.1 5.1 Priority Order

Engineering assumption, standard for self-consumption-first EMS design:

- On PV surplus: **PV** → **Load** first, then **PV** → **Battery**, then **PV** → **Grid (export)**.
- On PV deficit: **Battery** → **Load** first (battery is “free” marginal energy already paid for), then **Grid** → **Load** only once SOC_{min} is reached.

This ordering is appropriate because it maximizes self-consumption before falling back to the grid, which is the behaviour that generates ToD arbitrage savings in the first place — it is also the behaviour Phase 2’s LP is expected to *improve upon*, so Phase 1 must implement it honestly (not already partially optimized) to give a fair baseline.

6.2 5.2 Pseudocode

```

for k = 1 to N:
    net = P_pv(k) - P_load(k)
    if net >= 0:
        # PV surplus
        P_ch(k) = min(net, P_ch_max, (SOC_max - SOC(k-1)) * E_B / (eta_ch * dT))
        remainder = net - P_ch(k)
        P_grid_exp(k) = min(remainder, P_grid_exp_max)
        # if remainder still positive after export cap -> curtailment (Part 10)
        P_dis(k) = 0; P_grid_imp(k) = 0
    else:
        # PV deficit
        deficit = -net
        P_dis(k) = min(deficit, P_dis_max, (SOC(k-1) - SOC_min) * E_B * eta_dis / dT)
        remainder = deficit - P_dis(k)
        P_grid_imp(k) = min(remainder, P_grid_imp_max)
        P_ch(k) = 0; P_grid_exp(k) = 0
    SOC(k) = SOC(k-1) + dT/E_B * (eta_ch*P_ch(k) - P_dis(k)/eta_dis)

```

6.3 5.3 MATLAB Implementation

```

function [P_ch, P_dis, P_grid_imp, P_grid_exp, SOC] = rule_based_ems(P_pv, P_load,
    ↪ params)
% RULE_BASED_EMS Deterministic self-consumption-first dispatch (Phase 1 baseline).
%
% Inputs:
%   P_pv, P_load : [N x 1] kW profiles
%   params       : struct with fields
%                   E_B, eta_ch, eta_dis, dT, SOC0, SOC_min, SOC_max,
%                   P_ch_max, P_dis_max, P_grid_imp_max, P_grid_exp_max
% Outputs: [N x 1] dispatch vectors and SOC trajectory ([N+1 x 1], SOC(1)=SOC0)

N = length(P_pv);
P_ch = zeros(N,1); P_dis = zeros(N,1);
P_grid_imp = zeros(N,1); P_grid_exp = zeros(N,1);

```

```

SOC = zeros(N+1,1); SOC(1) = params.SOC0;

for k = 1:N
    net = P_pv(k) - P_load(k);
    if net >= 0
        % Headroom to SOC_max, expressed as a charge power limit
        headroom_kW = (params.SOC_max - SOC(k)) * params.E_B / (params.eta_ch *
            ↪ params.dT);
        P_ch(k) = min([net, params.P_ch_max, max(headroom_kW,0)]);
        remainder = net - P_ch(k);
        P_grid_exp(k) = min(remainder, params.P_grid_exp_max);
        % NOTE: if remainder > P_grid_exp_max, the surplus is curtailed.
        % Track this explicitly -- see Part 10 for a curtailment metric.
    else
        deficit = -net;
        headroom_kWh = max(SOC(k) - params.SOC_min, 0) * params.E_B;
        dis_limit_kW = headroom_kWh * params.eta_dis / params.dT;
        P_dis(k) = min([deficit, params.P_dis_max, dis_limit_kW]);
        remainder = deficit - P_dis(k);
        P_grid_imp(k) = min(remainder, params.P_grid_imp_max);
        % NOTE: if remainder > P_grid_imp_max, load is unserved --
        % flag as an infeasible scenario for this rule set (should not
        % occur if P_grid_imp_max is set realistically).
    end
    SOC(k+1) = SOC(k) + params.dT/params.E_B * ...
        (params.eta_ch*P_ch(k) - P_dis(k)/params.eta_dis);
end
end

```

7 Part 6 — Linear Programming EMS (Phase 2)

7.1 6.1 Decision Variables

For an N -step horizon, stack the decision vector as:

$$x = \begin{bmatrix} P_{grid,imp}(1:N) \\ P_{grid,exp}(1:N) \\ P_{ch}(1:N) \\ P_{dis}(1:N) \end{bmatrix} \in \mathbb{R}^{4N}$$

SOC is *not* an independent decision variable in this formulation — it is a linear function of P_{ch} , P_{dis} via the recursion in Part 3, so its bounds become linear inequality constraints on cumulative charge/discharge (Section 6.4). This halves the problem size versus including SOC explicitly, though the explicit-SOC form is also valid (**optional implementation choice**) if you find it more transparent for debugging.

7.2 6.2 Objective Function

$$\min_x f^T x = \sum_{k=1}^N C_{grid}(k) P_{grid,imp}(k) \Delta T - \sum_{k=1}^N C_{exp}(k) P_{grid,exp}(k) \Delta T$$

Battery degradation is deliberately **excluded** here per Phase 2's definition in `Plan.md` — this is the “naive” objective whose consequence (aggressive cycling) is quantified *after* solving, in Section 6.6, and is what motivates Phase 3.

7.3 6.3 Equality Constraints ($A_{eq}x = b_{eq}$)

One row per time step, from Part 4.2:

$$P_{grid,imp}(k) - P_{grid,exp}(k) + P_{dis}(k) - P_{ch}(k) = P_{load}(k) - P_{pv}(k)$$

7.4 6.4 Inequality Constraints ($Ax \leq b$) — SOC Bounds

Substituting the SOC recursion (Part 3.1) forward from $SOC(0)$:

$$SOC(k) = SOC(0) + \frac{\Delta T}{E_B} \sum_{j=1}^k \left(\eta_{ch} P_{ch}(j) - \frac{P_{dis}(j)}{\eta_{dis}} \right)$$

so $SOC_{min} \leq SOC(k) \leq SOC_{max}$ becomes two linear inequalities per step in cumulative sums of P_{ch} , P_{dis} — implemented in MATLAB as a running lower-triangular coefficient matrix (see code below).

7.5 6.5 Bounds ($lb \leq x \leq ub$)

$$\begin{aligned} 0 \leq P_{grid,imp}(k) \leq P_{grid,imp}^{max}, \quad 0 \leq P_{grid,exp}(k) \leq P_{grid,exp}^{max} \\ 0 \leq P_{ch}(k) \leq P_{ch,max}, \quad 0 \leq P_{dis}(k) \leq P_{dis,max} \end{aligned}$$

7.6 6.6 MATLAB: Building the LP for linprog

```

function lp = build_lp_problem(P_pv, P_load, tariff, params)
% BUILD_LP_PROBLEM Constructs f, A, b, Aeq, beq, lb, ub for linprog
% for the Phase-2 "naive" cost-minimizing EMS over an N-step horizon.
%
% tariff: struct with fields C_import [N x 1], C_export [N x 1]

N = length(P_pv);
dT = params.dT;

% Decision vector layout: [Pimp(1:N); Pexp(1:N); Pch(1:N); Pdis(1:N)]
idx.imp = 1:N;
idx.exp = N+1:2*N;
idx.ch = 2*N+1:3*N;
idx.dis = 3*N+1:4*N;
nvar = 4*N;

% ---- Objective f (linprog minimizes f'*x) ----
f = zeros(nvar,1);
f(idx.imp) = tariff.C_import(:) * dT;
f(idx.exp) = -tariff.C_export(:) * dT; % export revenue = negative cost

% ---- Equality constraints: power balance ----
Aeq = zeros(N, nvar);
beq = P_load(:) - P_pv(:);
for k = 1:N
    Aeq(k, idx.imp(k)) = 1;
    Aeq(k, idx.exp(k)) = -1;
    Aeq(k, idx.dis(k)) = 1;
    Aeq(k, idx.ch(k)) = -1;
end

% ---- Inequality constraints: SOC bounds via cumulative sums ----
% SOC(k) = SOC0 + dT/E_B * cumsum(eta_ch*Pch - Pdis/eta_dis)
L = tril(ones(N)); % lower-triangular cumulative-sum operator
coefCh = params.eta_ch * dT / params.E_B;
coefDis = -dT / (params.eta_dis * params.E_B);

A_upper = zeros(N, nvar); % SOC(k) <= SOC_max
A_upper(:, idx.ch) = coefCh * L;
A_upper(:, idx.dis) = coefDis * L;
b_upper = (params.SOC_max - params.SOC0) * ones(N,1);

A_lower = -A_upper; % SOC(k) >= SOC_min <=> -SOC(k) <= -SOC_min
b_lower = -(params.SOC_min - params.SOC0) * ones(N,1);

A = [A_upper; A_lower];
b = [b_upper; b_lower];

% ---- Bounds ----
lb = zeros(nvar,1);
ub = zeros(nvar,1);

```

```
ub(idx.imp) = params.P_grid_imp_max;
ub(idx.exp) = params.P_grid_exp_max;
ub(idx.ch)   = params.P_ch_max;
ub(idx.dis) = params.P_dis_max;

lp =
    ↳ struct('f',f,'A',A,'b',b,'Aeq',Aeq,'beq',beq,'lb',lb,'ub',ub,'idx',idx,'N',N);
end

function result = run_lp_ems(lp)
% RUN_LP_EMS Solves the LP built by build_lp_problem and unpacks results.
options = optimoptions('linprog','Display','none');
[x, fval, exitflag, output] = linprog(lp.f, lp.A, lp.b, lp.Aeq, lp.beq, lp.lb,
    ↳ lp.ub, options);

if exitflag ~= 1
    warning('linprog did not converge cleanly, exitflag=%d', exitflag);
end

result.P_grid_imp = x(lp.idx.imp);
result.P_grid_exp = x(lp.idx.exp);
result.P_ch       = x(lp.idx.ch);
result.P_dis      = x(lp.idx.dis);
result.cost       = fval;
result.exitflag   = exitflag;
result.output     = output;
end
```

8 Part 7 — Tariff Model

8.1 7.1 Design Requirement

The tariff model must accept a **price vector**, not hard-coded regulatory numbers, so the same LP code works for flat, ToD, or sensitivity-swept tariffs. **Project assumption:** APDCL-specific ToD slab values are *not* asserted in this blueprint — see Part 8 for why they must be independently verified before being used as fact in your report.

8.2 7.2 MATLAB Tariff Construction

```
function tariff = calculate_tariff(N, dT, mode, varargin)
% CALCULATE_TARIFF Builds C_import / C_export vectors of length N.
%
% mode = 'flat'      : varargin{1} = flat_price
% mode = 'tod'       : varargin{1} = struct with fields
%                       .peak_hours [start end] in 24h clock, .peak_price
%                       .offpeak_price, .normal_price
% mode = 'custom'    : varargin{1} = C_import vector directly

steps_per_hour = 1/dT;
hour_of_step = mod((0:N-1)/steps_per_hour, 24);

switch mode
    case 'flat'
        price = varargin{1};
        C_import = price * ones(N,1);
    case 'tod'
        p = varargin{1};
        C_import = p.normal_price * ones(N,1);
        is_peak = hour_of_step(:) >= p.peak_hours(1) & hour_of_step(:) <
            ⇨ p.peak_hours(2);
        C_import(is_peak) = p.peak_price;
        if isfield(p, 'offpeak_hours')
            is_off = hour_of_step(:) >= p.offpeak_hours(1) & hour_of_step(:) <
                ⇨ p.offpeak_hours(2);
            C_import(is_off) = p.offpeak_price;
        end
    case 'custom'
        C_import = varargin{1}(:);
    otherwise
        error('Unknown tariff mode');
end

% Export price: engineering assumption -- flat feed-in rate unless
% varargin{2} provides a vector (net metering typically credits at a
% single average/avoided-cost rate; EXIM metering may differ -- verify
% against Part 8 before finalizing).
if nargin >= 5
    C_export = varargin{2}(:);
else
    C_export = zeros(N,1); % conservative default: no export credit assumed
end
```

```
end
```

```
tariff.C_import = C_import;  
tariff.C_export = C_export;
```

```
end
```

Example construction (illustrative values only, not a regulatory claim):

```
p.normal_price = 6.5; p.peak_price = 9.0; p.offpeak_price = 4.5;  
p.peak_hours = [18 22]; p.offpeak_hours = [23 6];  
tariff = calculate_tariff(96, 0.25, 'tod', p);
```

9 Part 8 — Net vs. EXIM Metering Context

9.1 8.1 Electrical Architecture vs. Metering vs. Regulation

These are three **separate layers** and must not be conflated in your report:

1. **Electrical architecture:** where the meter sits relative to PV, load, battery, and grid. In a typical behind-the-meter installation, PV and battery sit *upstream* of a single bidirectional meter that only sees the *net* flow: $P_{grid}(k) = P_{load}(k) + P_{ch}(k) - P_{pv}(k) - P_{dis}(k)$ (consistent with Part 4's balance equation). The meter cannot distinguish "PV exporting" from "battery exporting" — it only sees net import/export.
2. **Metering arrangement (net vs. EXIM):**
 - **Net metering:** a single bidirectional meter nets consumption against generation over a billing period; only the *net* energy is billed/credited.
 - **EXIM (export/import) metering:** separate registers (or separate meters) record gross import and gross export independently, each billed/credited at its own applicable rate. This is the arrangement typically required once a battery is added behind the meter, because a bidirectional net meter cannot distinguish grid-charged battery export from PV-charged battery export, which regulators may treat differently.
3. **Tariff eligibility & regulatory approval:** whether a given consumer category (e.g., domestic rooftop solar) qualifies for net vs. EXIM metering, and under what capacity/ownership conditions, is set by the applicable state regulation (for Assam, the AERC/APDCL rooftop solar regulations) and **is not something this blueprint asserts**.

9.2 8.2 Behind-the-Meter Battery Effect on Metered Flow

Because the meter only sees the net terms in the equation above, a behind-the-meter battery **changes what the grid meter records** without changing the underlying physical PV generation — this is precisely why EXIM arrangements exist: to prevent a battery from effectively laundering grid-imported energy as "solar export" for credit purposes.

9.3 8.3 What You Must Verify Before Claiming Anything Regulatory

Before your report states which metering arrangement applies to your assumed system:

- Confirm the applicable **AERC (Assam Electricity Regulatory Commission)** rooftop solar / net-metering regulations currently in force, and whether they distinguish storage-coupled systems.
- Confirm **APDCL's** connection-agreement terms for the capacity range you assume.
- If you cannot verify these within project time, **state the metering arrangement as a project assumption** explicitly (e.g., "this project assumes a net-metering arrangement for simplicity; EXIM requirements were not verified

against current AERC regulation and are noted as a limitation”) rather than asserting it as fact. This is exactly the kind of claim Part 25’s honesty requirement is about.

10 Part 9 — Peak Demand Management (Optional Extension)

Project assumption: as noted in the Source & Scope Note, Plan.md does not define peak-shaving as a mandatory phase. This section is included so the capability exists if you choose to extend Phase 3, but it is not on the critical path.

10.1 9.1 Energy Charge vs. Demand Charge

- **Energy charge:** billed on total kWh consumed, at the ToD rate in force at each step — this is what Phases 2-3 already optimize.
- **Demand/peak charge:** billed on the single highest metered import power (kW) during a billing period, independent of how many kWh were consumed at that level. A single high-power spike can dominate a bill even if total energy is low.

10.2 9.2 LP-Compatible Peak Formulation

Introduce one auxiliary scalar P_{peak} and add N linear constraints:

$$P_{grid,imp}(k) \leq P_{peak} \quad \forall k$$

Objective becomes:

$$J = w_1 \sum_k C_{grid}(k) P_{grid,imp}(k) \Delta T + w_2 P_{peak}$$

This remains linear — P_{peak} is just one more decision variable appended to x , with one extra column in f (weight w_2) and N extra rows in A (one per step, coefficient +1 on $P_{grid,imp}(k)$ and -1 on P_{peak} , bound ≤ 0).

```
% Appended to build_lp_problem.m if peak-shaving is enabled:
% new variable index: idx.peak = nvar_old + 1
% f(idx.peak) = w2;
% A_peak = zeros(N, nvar_old+1);
% for k=1:N
%     A_peak(k, idx.peak) = 1;
%     A_peak(k, idx.peak) = -1;
% end
% b_peak = zeros(N,1);
% A = [A; A_peak]; b = [b; b_peak];
```

10.3 9.3 Comparison Ladder

Plan.md's comparison matrix (Phase 4) is naturally extended to include peak-aware variants if implemented: PV-only → PV+rule-based → PV+cost-optimized → PV+cost+peak-optimized, each adding one more term to the objective while reusing the same constraint machinery.

11 Part 10 — PV Self-Consumption / Utilization Metrics

11.1 10.1 Definitions

$$\text{Self-consumption} = \frac{\sum_k (P_{pv}(k) - P_{grid,exp}(k) - P_{curt}(k)) \Delta T}{\sum_k P_{pv}(k) \Delta T}$$

Fraction of generated PV energy actually used on-site (directly or via battery), rather than exported or curtailed.

$$\text{Self-sufficiency} = \frac{\sum_k (P_{load}(k) - P_{grid,imp}(k)) \Delta T}{\sum_k P_{load}(k) \Delta T}$$

Fraction of load energy met without grid import.

$$\text{PV utilization} = \frac{\sum_k P_{pv}(k) \Delta T - \sum_k P_{curt}(k) \Delta T}{\sum_k P_{pv}(k) \Delta T}$$

Fraction of available PV energy not curtailed.

Curtailement $P_{curt}(k)$ only arises if, at a given step, PV surplus exceeds both the battery's charge headroom *and* the grid export limit (Part 5.3's rule-based logic flags this case; in the LP formulation it appears implicitly as an infeasibility unless $P_{curt}(k)$ is added as an explicit slack variable — recommended so the LP always stays feasible).

11.2 10.2 MATLAB

```
function m = calculate_metrics(P_pv, P_load, P_grid_imp, P_grid_exp, P_curt, dT)
E_pv = sum(P_pv)*dT;
E_load = sum(P_load)*dT;
m.self_consumption = (E_pv - sum(P_grid_exp)*dT - sum(P_curt)*dT) / E_pv;
m.self_sufficiency = (E_load - sum(P_grid_imp)*dT) / E_load;
m.pv_utilization = (E_pv - sum(P_curt)*dT) / E_pv;
m.grid_import_kWh = sum(P_grid_imp)*dT;
m.grid_export_kWh = sum(P_grid_exp)*dT;
end
```

12 Part 11 — Battery Degradation

12.1 11.1 Choosing a Model

Approach	Complexity	Fit for this project
Throughput-based (Ah/kWh cycled)	Low	Simple but ignores DoD sensitivity
Equivalent full cycles (EFC)	Low-Medium	Good balance; standard in EMS literature
DoD-based life curve	Medium	Recommended primary approach
Rainflow counting	Medium-High	More rigorous for irregular cycling; good as a validation cross-check on the naive-vs-aware comparison <code>Plan.md</code> already calls for
Semi-empirical electrochemical aging	High	Out of scope — flag as future work only

Recommendation: use a **DoD-based cycle-life stress function** as the primary degradation cost inside the optimization (needed at every LP iteration, so it must stay simple and ideally linear/quadratic), and use **rainflow counting as a post-hoc validation** of the DoD-based estimate on the resulting dispatch — this is exactly the two-stage structure `Plan.md`’s Phase 2 already specifies (`linprog` first, rainflow-based degradation analysis after).

12.2 11.2 Degradation Cost Formulation

Battery cycle life $LifeCycles(DoD)$ decreases sharply as DoD increases — a standard **literature-based assumption**, commonly modeled as a power-law:

$$LifeCycles(DoD) = A \cdot DoD^{-b}$$

with A, b manufacturer/literature-derived constants — [INSERT VALUE] once you select a specific cell chemistry/datasheet.

Per-step degradation cost, from `Plan.md`’s own formulation:

$$C_{deg}(k) = Cost_{replacement} \times \frac{|P_{batt}(k)| \Delta T}{2 E_B LifeCycles(DoD(k))}$$

The factor of 2 converts a half-cycle (charge or discharge alone) throughput into an equivalent full-cycle fraction.

12.3 11.3 Linearizing for LP/QP Compatibility

$LifeCycles(DoD)^{-1}$ is nonlinear in DoD. Two LP/QP-compatible options:

- **Piecewise-linear approximation** of $1/LifeCycles(DoD)$ over a few DoD bins (e.g., 0-20%, 20-50%, 50-80%, 80-100%), each with its own slope — keeps the problem an LP with a few extra SOS2/piecewise constraints, or can be approximated with a single conservative slope for simplicity (**engineering assumption**, adequate for a first B.Tech pass).
- **Quadratic penalty** on throughput, $C_{deg}(k) \approx \beta (P_{ch}(k) + P_{dis}(k))^2$, which fits directly into quadprog and qualitatively captures “large power swings cost disproportionately more” without needing an explicit DoD curve. This is the **recommended primary approach** for Phase 3, since it keeps solver family continuity with quadprog.

12.4 11.4 MATLAB

```
function [C_deg, EFC] = calculate_degradation(P_ch, P_dis, dT, E_B,
    ↪ cost_replacement, lifecycle_fn)
% CALCULATE_DEGRADATION Post-simulation throughput-based degradation cost.
% lifecycle_fn: function handle, LifeCycles(DoD) -> cycles, e.g. @(d) A*d.^(-b)

throughput_kWh = (P_ch + P_dis) * dT;      % energy cycled through the battery
EFC = sum(throughput_kWh) / (2*E_B);      % equivalent full cycles

% Engineering assumption: approximate per-step DoD as instantaneous
% cycling depth P*dT/E_B; for a rigorous rainflow-based DoD, post-process
% the SOC trajectory with a rainflow counting utility instead.
DoD_est = throughput_kWh / E_B;
DoD_est(DoD_est < 1e-6) = 1e-6; % avoid division issues at zero cycling

life = lifecycle_fn(DoD_est);
C_deg_per_step = cost_replacement * throughput_kWh ./ (2 * E_B * life);
C_deg = sum(C_deg_per_step);
end
```

13 Part 12 — Multi-Objective Formulation (Phase 3)

13.1 12.1 Combined Objective

From Plan.md, directly:

$$\min_x \sum_{k=1}^N \left[C_{grid}(k) P_{grid,imp}(k) \Delta T - C_{exp}(k) P_{grid,exp}(k) \Delta T + \alpha C_{deg}(P_{batt}(k), SOC(k)) \right]$$

13.2 12.2 Normalization and Weighting

C_{grid} terms are in currency units already; C_{deg} (Part 11) is also currency, so α is **dimensionless** here — a meaningful simplification versus the more general multi-objective case (Part 12.3) where terms may have different units. **Recommendation:** start with $\alpha = 1$ (both terms weighted equally in currency terms) and run an α -sweep (Part 14) to show the cost/degradation trade-off curve — this sweep is the Phase 3 deliverable Plan.md asks for, not a single fixed value.

13.3 12.3 General Weighted-Sum Form (if peak/export terms are added)

$$J = w_1 C_{energy} + w_2 C_{peak} + w_3 C_{degradation} + w_4 C_{export}$$

If C_{peak} (kW) and C_{energy} (currency) have different units, each term must first be normalized (e.g., divide by its Phase-2 baseline value) before combining, otherwise w_i absorbs unit conversion invisibly and loses physical meaning. **Recommendation for this project:** keep the objective to the two terms in Section 12.1 (C_{grid} , C_{deg}) as the primary Phase 3 deliverable; treat the 4-term form as an **optional implementation choice** only if Part 9's peak-shaving extension is also implemented.

13.4 12.4 Solving with quadprog

```
function result = run_degradation_aware_ems(P_pv, P_load, tariff, params, alpha,
    beta)
% Quadratic degradation penalty: C_deg_approx(k) = beta*(Pch(k)+Pdis(k))^2
% H encodes the quadratic term; f encodes the linear (grid cost) term.
lp = build_lp_problem(P_pv, P_load, tariff, params); % reuse Aeq/A/b/lb/ub
nvar = length(lp.f);
H = zeros(nvar);
for k = 1:lp.N
    ich = lp.idx.ch(k); idis = lp.idx.dis(k);
    % (Pch+Pdis)^2 = Pch^2 + 2 Pch Pdis + Pdis^2
    H(ich,ich) = H(ich,ich) + 2*alpha*beta;
    H(idis,idis) = H(idis,idis) + 2*alpha*beta;
    H(ich,idis) = H(ich,idis) + 2*alpha*beta;
    H(idis,ich) = H(idis,ich) + 2*alpha*beta;
end
options = optimoptions('quadprog','Display','none');
[x, fval, exitflag] = quadprog(H, lp.f, lp.A, lp.b, lp.Aeq, lp.beq, lp.lb, lp.ub,
    [], options);
```

```
result.P_grid_imp = x(lp.idx.imp); result.P_grid_exp = x(lp.idx.exp);  
result.P_ch = x(lp.idx.ch); result.P_dis = x(lp.idx.dis);  
result.cost = fval; result.exitflag = exitflag;  
end
```

Optional implementation choice: if you prefer the piecewise-linear DoD-based cost from Part 11.3 instead of the quadratic proxy, the problem stays an LP (linprog with extra piecewise columns) rather than moving to quadprog — either is defensible; document which you chose and why in your methodology chapter.

14 Part 13 — Scenario Analysis (Phase 4)

14.1 13.1 Scenario Matrix

Scenario axis	Levels	Purpose	Essential?
Solar	Clear day, cloudy day, low-generation day	Test dispatch stability under irradiance variability	Essential
Load	Normal, high, evening-heavy	Test EMS behaviour when demand peak coincides with/without PV	Essential
Battery size	Small, medium, large	Show diminishing/increasing returns with capacity (feeds Part 14)	Essential
Tariff	Flat, ToD, tariff-sensitivity sweep	Show arbitrage value depends on tariff spread	Essential

Scenario	Purpose	Inputs	Expected behaviour	Metrics	Expected plot
Clear day	Best-case PV	High, smooth PV curve	Strong midday charging, minimal grid import	Self-sufficiency, grid cost	Power balance vs. time
Cloudy day	Stress test PV intermittency	Noisy/low PV curve	More grid reliance, shallower battery cycles	Grid import kWh	PV vs. grid import overlay
Evening-heavy load	PV/load mismatch	Load peak after sunset	Battery discharges to cover evening peak; ToD savings depend on evening tariff	Peak grid import, cost	SOC vs. time

Scenario	Purpose	Inputs	Expected behaviour	Metrics	Expected plot
Small vs. large battery	Capacity sensitivity	Same PV/load, E_B varied	Diminishing marginal savings per added kWh	Savings vs. E_B	Part 14 sensitivity curve
Flat vs. ToD tariff	Tariff-value isolation	Same PV/load/battery	ToD case shows larger savings than flat	Cost delta	Bar chart comparison

Combinatorial testing of all axes simultaneously is not required — the essential set above (one axis varied at a time against a fixed baseline) is sufficient for a B.Tech project and keeps run count manageable; full factorial combination is listed as **optional** if time permits.

15 Part 14 — Sensitivity Analysis

15.1 14.1 Parameters to Sweep

Battery capacity, battery power rating, battery replacement cost, tariff spread (peak–offpeak), round-trip efficiency, SOC limits, PV capacity, load magnitude.

15.2 14.2 Recommended Plots

- Battery capacity vs. annualized savings
- Battery capacity vs. peak reduction (if Part 9 implemented)
- Battery capacity vs. battery throughput (EFC)
- Tariff spread vs. optimal battery utilization
- α (Part 12) vs. grid cost / degradation cost trade-off

15.3 14.3 MATLAB: Automated Sweep

```
function results = run_sensitivity_sweep(param_name, param_values, P_pv, P_load,
    ↪ tariff, base_params)
% RUN_SENSITIVITY_SWEEP Re-runs the LP EMS for each value of one parameter.
results = struct('value', {}, 'cost', {}, 'EFC', {}, 'self_sufficiency', {});
for i = 1:length(param_values)
    p = base_params;
    p.(param_name) = param_values(i);
    lp = build_lp_problem(P_pv, P_load, tariff, p);
    r = run_lp_ems(lp);
    m = calculate_metrics(P_pv, P_load, r.P_grid_imp, r.P_grid_exp,
    ↪ zeros(size(P_pv)), p.dT);
    [~, EFC] = calculate_degradation(r.P_ch, r.P_dis, p.dT, p.E_B,
    ↪ p.cost_replacement, p.lifecycle_fn);

    results(i).value = param_values(i);
    results(i).cost = r.cost;
    results(i).EFC = EFC;
    results(i).self_sufficiency = m.self_sufficiency;
end
end
```

16 Part 15 — Model Predictive Control (Phase 5)

16.1 15.1 Formulation

Element	Definition
Prediction horizon	24 h ahead (96 steps at $\Delta T=0.25\text{h}$)
Control horizon	1 step applied per re-optimization (standard receding-horizon MPC)
State	$SOC(k)$
Inputs (decision)	$P_{ch}(k), P_{dis}(k), P_{grid,imp}(k), P_{grid,exp}(k)$ over the horizon
Disturbances	Forecast P_{pv}, P_{load} over the horizon (Part 16)
Constraints	Same as Phase 2/3 LP, re-applied at each horizon window
Objective	Same as Phase 2/3, evaluated over the rolling horizon

16.2 15.2 Day-Ahead LP vs. Receding-Horizon MPC

Phase 2's LP solves **once** for the entire day using perfect (or day-ahead-forecast) PV/load knowledge. MPC instead re-solves the **same LP structure** every ΔT , using only the *first* step of each solution, then slides the horizon forward and re-forecasts — this is what allows it to correct for forecast error as better information arrives, at the cost of N times more solves per day.

16.3 15.3 Pseudocode

```

SOC_current = SOC0
for t = 1 to N_sim:
    forecast_pv, forecast_load = get_forecast(t, horizon=96) # Part 16
    lp = build_lp_problem(forecast_pv, forecast_load, tariff_window, params, SOC0=SOC_curr
    result = run_lp_ems(lp)
    # apply ONLY the first control step
    apply(P_ch(1), P_dis(1), P_grid_imp(1), P_grid_exp(1))
    SOC_current = update_SOC(SOC_current, result step 1 values)
    log(t, applied values, SOC_current)

```

16.4 15.4 MATLAB Boilerplate

```

function log = run_mpc(P_pv_actual, P_load_actual, forecast_fn, tariff, params,
    ↪ horizon)
% RUN_MPC Receding-horizon controller reusing build_lp_problem/run_lp_ems.
N_sim = length(P_pv_actual);
SOC_current = params.SOC0;
log.P_ch = zeros(N_sim,1); log.P_dis = zeros(N_sim,1);
log.P_grid_imp = zeros(N_sim,1); log.P_grid_exp = zeros(N_sim,1);

```

```

log.SOC = zeros(N_sim+1,1); log.SOC(1) = SOC_current;

for t = 1:N_sim
    h = min(horizon, N_sim - t + 1);
    [fc_pv, fc_load] = forecast_fn(t, h);           % Part 16
    p = params; p.SOC0 = SOC_current;
    tw.C_import = tariff.C_import(t:t+h-1);
    tw.C_export = tariff.C_export(t:t+h-1);

    lp = build_lp_problem(fc_pv, fc_load, tw, p);
    r = run_lp_ems(lp);

    % apply only first step
    log.P_ch(t) = r.P_ch(1); log.P_dis(t) = r.P_dis(1);
    log.P_grid_imp(t) = r.P_grid_imp(1); log.P_grid_exp(t) = r.P_grid_exp(1);

    SOC_current = SOC_current + p.dT/p.E_B * ...
        (p.eta_ch*r.P_ch(1) - r.P_dis(1)/p.eta_dis);
    log.SOC(t+1) = SOC_current;
end
end

```

17 Part 16 — Forecast Error

17.1 16.1 Error Model

$$P_{pv}^{forecast}(k) = P_{pv}^{actual}(k) (1 + \varepsilon_{pv}(k)), \quad \varepsilon_{pv}(k) \sim \mathcal{N}(0, \sigma_{pv}^2)$$

$$P_{load}^{forecast}(k) = P_{load}^{actual}(k) (1 + \varepsilon_{load}(k)), \quad \varepsilon_{load}(k) \sim \mathcal{N}(0, \sigma_{load}^2)$$

Literature-based assumption: σ_{pv} typically grows with forecast horizon (near-term forecasts are more accurate); a simple defensible choice is $\sigma_{pv}(h) = \sigma_{pv,0} \sqrt{h/h_{max}}$, increasing from a small near-term value toward a larger day-ahead value. Exact σ values are project-configurable, not literature facts — treat them as swept parameters (Part 16.2), not asserted truths.

17.2 16.2 Comparison Protocol

Run MPC three times with identical actual profiles but different σ : **perfect forecast** ($\sigma = 0$), **moderate** ($\sigma \approx 10\text{--}15\%$), **high** ($\sigma \approx 25\text{--}30\%+$) — values are engineering assumptions to be tuned, not fixed literature constants. Evaluate cost, peak demand, SOC trajectory deviation, grid import, and battery throughput (EFC) across the three.

```
function [fc_pv, fc_load] = forecast_with_error(P_pv_actual, P_load_actual, t, h,
    ↪ sigma_pv, sigma_load)
window = t:min(t+h-1, length(P_pv_actual));
n = length(window);
fc_pv = P_pv_actual(window) .* (1 + sigma_pv*randn(n,1));
fc_load = P_load_actual(window) .* (1 + sigma_load*randn(n,1));
fc_pv = max(fc_pv, 0); % PV forecast cannot be negative
fc_load = max(fc_load, 0); % nor can load
end
```

18 Part 17 — MATLAB Project Structure

```
/data
    pv_profile.csv
    load_profile.csv
    tariff_profile.csv
/models
    system_params.m           % central parameter struct definition
/ems
    rule_based_ems.m
/optimization
    build_lp_problem.m
    run_lp_ems.m
    run_degradation_aware_ems.m
/degradation
    calculate_degradation.m
/mpc
    run_mpc.m
    forecast_with_error.m
/analysis
    calculate_metrics.m
    calculate_tariff.m
    run_sensitivity_sweep.m
    validate_power_balance.m
/plots
    plot_results.m
/dashboard
    ems_dashboard.mlapp
/main
    main_simulation.m         % top-level script tying everything together
```

Each folder maps 1:1 onto a phase or cross-cutting concern from Part 1’s dependency graph, so `main_simulation.m` should simply call functions from each folder in dependency order rather than containing algorithmic logic itself — this keeps the codebase modular and each function independently testable (Part 20).

19 Part 18 — Data

19.1 18.1 Minimum Required Data

PV generation profile, load profile, tariff profile — all as N -length vectors at the chosen ΔT .

19.2 18.2 Sources

Type	Description	Status
Real measured	Campus smart-meter logs, on-site irradiance sensor	Only usable if actually obtained — do not assume access without confirming with your department/lab
Literature-derived	Typical PV generation shapes from published irradiance data for the Guwahati/Assam region, typical Indian residential/institutional load curves	Usable with citation; still not “your” measured data
Synthetic	Parametric PV model (e.g., clear-sky irradiance $\times$ panel efficiency) and load model (base load + occupancy-driven peaks)	Recommended fallback — always available, fully under your control, good for controlled scenario testing (Part 13)

19.3 18.3 Defensible Synthetic Profile Generation

```
function P_pv = generate_pv_profile(N, dT, P Rated, day_type)
% Simple clear-sky-like bell curve, modulated by day_type for scenario testing.
t_hours = (0:N-1)*dT;
sunrise = 6; sunset = 18; solar_noon = 12;
shape = max(0, cos(pi/2 * (t_hours - solar_noon)/(sunset - solar_noon)) .* ...
    (t_hours > sunrise & t_hours < sunset));
switch day_type
    case 'clear', scale = 1.0; noise = 0.02;
    case 'cloudy', scale = 0.55; noise = 0.20;
    case 'low', scale = 0.25; noise = 0.30;
    otherwise, error('unknown day_type');
end
P_pv = P Rated * scale * shape(:) .* (1 + noise*randn(N,1));
P_pv = max(P_pv, 0);
end

function P_load = generate_load_profile(N, dT, base_kW, peak_kW, load_type)
t_hours = (0:N-1)*dT;
switch load_type
```

```
case 'normal'
    peak_centers = [8 19]; widths = [1.5 2.5];
case 'high'
    peak_centers = [8 19]; widths = [2 3]; peak_kW = peak_kW*1.3;
case 'evening_heavy'
    peak_centers = [19]; widths = [3]; peak_kW = peak_kW*1.5;
otherwise, error('unknown load_type');
end
P_load = base_kW * ones(N,1);
for i = 1:length(peak_centers)
    P_load = P_load + (peak_kW-base_kW) * ...
        exp(-((t_hours(:)-peak_centers(i)).^2)/(2*widths(i)^2));
end
end
```

These are clearly labeled as synthetic, parametric profiles — appropriate for demonstrating EMS *behaviour*, but your report must not present their absolute numeric outputs as measured facts about any real installation.

Recommendation: treat full Simulink implementation as valuable for the rule-based (Phase 1) and MPC (Phase 5) demonstrations specifically, since these are the two phases with genuine closed-loop/dynamic character; the LP/QP-only phases (2-3) are naturally batch optimizations and can remain pure MATLAB scripts feeding results into Simulink-generated plots for consistency, if time is limited.

21 Part 20 — Validation

21.1 20.1 Checks

- Power balance residual ≈ 0 at every step (Part 4.2, within numerical tolerance).
- $SOC(k) \in [SOC_{min}, SOC_{max}] \forall k$.
- $P_{ch}(k), P_{dis}(k)$ within rated limits; $\min(P_{ch}(k), P_{dis}(k)) \approx 0$ (Part 3.2 check).
- $P_{grid,imp}(k), P_{grid,exp}(k) \geq 0$ and within connection limits.
- Energy conservation over the full horizon: total energy in = total energy out + stored change.
- Cost calculation consistency: manually recomputed cost from dispatch matches solver's fval.

21.2 20.2 MATLAB

```
function report = validate_power_balance(P_pv, P_load, r, params, tol)
if nargin < 5, tol = 1e-6; end
N = length(P_pv);
residual = r.P_grid_imp - r.P_grid_exp + r.P_dis - r.P_ch - (P_load - P_pv);
report.max_balance_residual = max(abs(residual));
report.balance_ok = report.max_balance_residual < tol;

SOC = zeros(N+1,1); SOC(1) = params.SOC0;
for k = 1:N
    SOC(k+1) = SOC(k) + params.dT/params.E_B * ...
        (params.eta_ch*r.P_ch(k) - r.P_dis(k)/params.eta_dis);
end
report.SOC = SOC;
report.soc_ok = all(SOC >= params.SOC_min - tol) && all(SOC <= params.SOC_max +
    ↪ tol);

report.simultaneous_cycling_max = max(min(r.P_ch, r.P_dis));
report.simultaneous_ok = report.simultaneous_cycling_max < tol;

report.bounds_ok = all(r.P_ch >= -tol) && all(r.P_ch <= params.P_ch_max + tol) &&
    ↪ ...
        all(r.P_dis >= -tol) && all(r.P_dis <= params.P_dis_max + tol);

report.all_pass = report.balance_ok && report.soc_ok && ...
    report.simultaneous_ok && report.bounds_ok;
end
```

Run this after **every** phase's dispatch (rule-based, naive LP, degradation-aware, MPC step) as a standard sanity gate before trusting any downstream metric or plot.

22 Part 21 — Expected Results (No Fabricated Numbers)

For each phase, generate the following and evaluate against the stated expectation — do not fill in numeric values until you have actually run the simulation.

Phase	Plot	Expected behaviour	Bug indicator	Meaningful improvement
Phase 1	PV, load, battery, grid power vs. time	Battery charges on PV surplus, discharges on deficit; grid only fills residual	Simultaneous charge+discharge; SOC outside bounds	N/A (baseline)
Phase 2	Dispatch + cost vs. Phase 1	Lower total grid cost than Phase 1; battery cycles more aggressively, timed to tariff peaks	Cost <i>higher</i> than Phase 1 (solver/constraint bug); SOC violations	Cost reduction of [INSERT VALUE]% vs. baseline
Phase 2 (post-hoc)	Degradation cost vs. Phase 1	Naive LP shows <i>higher</i> degradation cost than Phase 1 despite lower grid cost	Degradation cost lower than Phase 1 (would contradict the phase's own hypothesis — recheck model)	N/A — this contrast is the expected finding
Phase 3	Grid cost vs. degradation cost trade-off (α sweep)	Monotonic-ish trade-off: as α increases, degradation cost falls, grid cost rises	Non-monotonic/erratic curve (solver or normalization issue)	A visible “knee” showing a good compromise α

Phase	Plot	Expected behaviour	Bug indicator	Meaningful improvement
Phase 4	Comparison table (all controllers)	Ordering matches Plan.md's expected matrix: naive lowest grid cost/highest degradation; aware = lowest total cost	Any controller strictly dominates the LP-optimal ones on total cost (implies bug)	Clear demonstration that "optimal" $\neq$ "cheapest grid bill alone"
Phase 5	MPC vs. day-ahead LP	MPC slightly worse than day-ahead LP under perfect forecast (it has less foresight); gap widens with forecast error	MPC infeasible at some step; MPC <i>better</i> than perfect-information day-ahead LP (impossible — recheck)	Small, explainable gap that grows gracefully with σ

23 Part 22 — Report Boilerplate

The following are drafting templates, written in thesis register with explicit placeholders. Replace bracketed placeholders; do not present them as findings.

1. Introduction. Rising rooftop PV adoption in grid-connected residential and institutional settings has created a need for intelligent local energy management that goes beyond simple self-consumption logic. This project addresses the design of an Energy Management System (EMS) for a grid-tied PV-battery microgrid that explicitly accounts for both electricity tariff structure and battery degradation cost, rather than optimizing electricity cost in isolation.

2. Problem statement. A battery operated purely to minimize grid electricity cost under a Time-of-Day tariff can be cycled aggressively enough to shorten its service life disproportionately to the savings achieved, an effect that a cost-only optimization does not capture. This project quantifies that effect and develops a degradation-aware alternative.

3. Objectives. (i) Develop a rule-based baseline EMS. (ii) Formulate and solve a cost-minimizing LP dispatch. (iii) Quantify the battery degradation implied by the cost-only dispatch. (iv) Formulate a degradation-aware optimization balancing both objectives. (v) Benchmark all controllers across representative scenarios. (vi) Extend to a rolling-horizon MPC formulation under forecast uncertainty. (vii) [INSERT: Simulink/dashboard objective if implemented].

4. System architecture. [INSERT FIGURE: system architecture per Part 19.2]. The modeled system comprises a PV array, a grid-tied inverter, a battery energy storage system, and a bidirectional grid connection, coordinated by an EMS operating at a [INSERT VALUE]-minute control interval.

5. Mathematical modelling. [Insert Part 2–4 content, condensed to report length].

6. EMS methodology. Three control strategies are developed and compared: a deterministic rule-based controller (Section 7), a cost-minimizing linear program (Section 8), and a degradation-aware extension (Section 10).

7. Rule-based control. [Insert Part 5 content].

8. LP optimization. [Insert Part 6 content].

9. Peak shaving. *(Include only if implemented per Part 9.)* [Insert Part 9 content].

10. Battery degradation. [Insert Parts 11–12 content].

11. MPC. [Insert Part 15 content].

12. Simulation methodology. All controllers are evaluated on [INSERT: real / synthetic] PV and load profiles at a [INSERT VALUE]-minute resolution over [INSERT VALUE]-day scenarios described in Section 13.

13. Performance metrics. Grid energy cost, battery degradation cost (equivalent full cycles and monetary estimate), self-consumption, self-sufficiency, and [INSERT: peak demand, if implemented] are used consistently across all controllers (Part 10, Part 20).

14. Results discussion template. *Structure each results subsection as:* (a) [INSERT FIGURE]; (b) numerical summary table [INSERT VALUE]s; (c) interpretation against the expected-behaviour criteria of Part 21; (d) any deviation from expectation and its likely cause.

15. Limitations. The degradation model used is a [INSERT: throughput-based / DoD-based / quadratic-proxy] approximation and does not capture calendar aging, temperature effects, or manufacturer-specific cell chemistry in detail. The metering arrangement assumed (Section 8) has not been independently verified against current AERC/APDCL regulation. PV and load data are [INSERT: real / synthetic] and results should be interpreted accordingly.

16. Future work. Rainflow-based degradation validation at full resolution; incorporation of temperature-dependent aging; real smart-meter data acquisition; meta-heuristic (PSO/GA) comparison against the QP-based Phase 3 solution as an alternative solver benchmark; full hardware-in-the-loop or physical prototype validation.

17. Conclusion. [INSERT: summary of the cost/degradation trade-off finding once simulated, referencing the α -sweep of Section 10].

24 Part 23 — Figure and Plot Plan

#	Title	Shows	Why it matters	Generating script
1	System architecture	Block diagram of PV/battery/grid/load/EMS	Orients the reader before and after	(manual/drawn)
2	PV & load profile	Input profiles used	Establishes the scenario being solved	plot_results.m
3	Rule-based dispatch	P_{pv} , P_{load} , P_{batt} , P_{grid} vs. time	Baseline behaviour	plot_results.m
4	LP (naive) dispatch	Same, Phase 2	Shows cost-driven cycling pattern	plot_results.m
5	SOC comparison	SOC(k) across controllers	Visualizes cycling depth/frequency differences	plot_results.m
6	Grid power comparison	$P_{grid,imp/exp}$ across controllers	Shows arbitrage behaviour	plot_results.m
7	Tariff profile	$C_{import}(k)$, $C_{export}(k)$	Context for dispatch timing	calculate_tariff.m output
8	Peak shaving (if implemented)	$P_{grid,imp}$ with/without peak cap	Demand-charge effect	plot_results.m
9	PV utilization	Self-consumption/curtailment breakdown	Quantifies PV value capture	calculate_metrics.m output
10	Degradation cost trade-off	Grid cost vs. degradation cost, α -swept	Core Phase 3 finding	run_sensitivity_sweep.m
11	Sensitivity: battery capacity vs. savings	Line plot	Justifies sizing discussion	run_sensitivity_sweep.m
12	MPC vs. day-ahead LP	Dispatch + cost comparison	Validates Phase 5	run_mpc.m output
13	Forecast error comparison	Cost/peak/SOC vs. σ	Quantifies robustness	forecast_with_error.m sweep

25 Part 24 — Viva Preparation

Short project explanation (1-2 minutes): “This project designs an energy management system for a grid-connected solar PV and battery microgrid. Rather than only minimizing the electricity bill under a Time-of-Day tariff, the EMS is designed to also account for the cost of battery degradation caused by aggressive cycling. I start with a simple rule-based controller as a baseline, then formulate a cost-minimizing linear program, show that this naive optimization saves money but wears the battery faster, and then develop a degradation-aware formulation that balances both. I benchmark all three approaches across representative solar, load, and tariff scenarios, and extend the day-ahead optimization into a rolling-horizon Model Predictive Control loop that re-optimizes as forecasts update.”

Likely questions and answers:

1. *Why is a battery needed at all, given the grid connection?* — It enables ToD arbitrage (shift consumption away from peak-price hours) and improves self-consumption of PV that would otherwise be exported at a lower credit rate than the avoided import cost.
2. *What is a Time-of-Day tariff and why does it matter here?* — A tariff structure where the import price varies by hour of day; it is what creates the economic incentive for the battery to time-shift energy, which the LP in Phase 2 exploits.
3. *What’s the difference between net metering and EXIM metering?* — See Part 8: net metering bills only the net of import/export; EXIM meters/bills import and export separately, relevant once a battery can obscure the source of exported energy.
4. *Why does the naive LP degrade the battery faster than the rule-based baseline?* — Because it is not penalized for cycling frequency/depth at all — any arbitrage opportunity, however small, is exploited, including shallow cycles chasing minor price differences that a degradation-aware objective would judge not worth the wear.
5. *Why LP for Phase 2 and QP (or piecewise-LP) for Phase 3?* — The grid-cost objective and constraint set are linear; the recommended quadratic degradation proxy (Part 11.3) requires a quadratic solver, so quadprog is the natural progression while remaining in the convex-optimization family (as opposed to jumping to non-convex metaheuristics).
6. *What does α control, and how did you choose it?* — It weights degradation cost against grid cost in the combined objective; rather than a single fixed value, an α -sweep is used to show the trade-off curve and justify a chosen operating point.
7. *How do you know the battery isn’t charging and discharging at the same time?* — Explicit post-solve validation (Part 3.2, Part 20) checks $\min(P_{ch}(k), P_{dis}(k)) \approx 0$ at every step; round-trip losses make simultaneous cycling suboptimal for a cost-minimizing solver in the first place.
8. *Why MPC instead of just solving the day-ahead LP once?* — Day-ahead LP assumes perfect knowledge of the full day’s PV/load in advance; MPC instead re-optimizes on a rolling basis using only near-term forecasts, which is more realistic and allows the controller to correct for forecast error as the day progresses.
9. *What assumptions most affect your results?* — Efficiency values (η_{ch}, η_{dis}), SOC limits, the degradation model form (quadratic proxy vs. DoD curve), and whether

PV/load data are synthetic or measured — all explicitly flagged in Sections 2, 11, and 18.

10. *What are the limits of your degradation model?* — It does not capture calendar aging, temperature dependence, or chemistry-specific nonlinearities; it is a defensible first-order approximation appropriate to project scope, not a production-grade battery management model.
11. *Why not just use PSO or a neural network for everything?* — Convex LP/QP formulations guarantee global optimality and fast, reliable solves suitable for the rolling MPC loop; metaheuristics are listed only as an optional comparison, not the primary method, to keep the project's core defensible and its solve times tractable.
12. *How would this change with a bigger/smaller battery?* — Explored directly in the Part 14 sensitivity sweep — larger batteries generally increase achievable savings with diminishing marginal returns, at higher capital and degradation-exposure cost.
13. *What happens if PV forecast is wrong?* — Quantified in Part 16 — cost and peak-demand outcomes degrade gracefully as forecast error σ increases, evaluated by re-running the MPC loop under different error levels.
14. *Is this project novel?* — See Part 25 — the individual optimization techniques are established; the project's contribution is the integrated degradation-aware EMS framework applied to a specific tariff/regional context, evaluated across a defined scenario matrix.
15. *What would you do differently with more time?* — [INSERT: your own honest answer once you've built the project — e.g., real data acquisition, rainflow-validated degradation model, hardware prototype].

26 Part 25 — What Is Actually Novel?

Honest assessment: this project is primarily **implementation-oriented and comparative**, not a novel algorithmic contribution. Linear programming for EMS dispatch, DoD-based battery degradation modeling, and receding-horizon MPC are all well established individually in the microgrid EMS literature. That is normal and appropriate for a B.Tech final-year project — the value is in a **correctly implemented, validated, and honestly evaluated integrated system**, not in inventing a new optimization algorithm.

Realistic ways to build a defensible small contribution within this project:

- Explicit **cost-vs-degradation trade-off quantification** via the α -sweep (Section 12), presented as a decision-support curve rather than a single answer — this framing itself is a useful, presentable contribution even though the underlying LP/QP methods are standard.
- **Naive-vs-aware comparison as an explicit demonstrated finding** (Phase 2 → Phase 3 narrative from Plan.md) — showing *quantitatively* that cost-only optimization is a false economy for battery life is a concrete, defensible result rather than a restatement of known theory.
- Application to an **Assam/APDCL tariff and regional PV/load context**, if you obtain or credibly approximate regional data — regional grounding is a legitimate (if modest) contribution axis.
- **Forecast-uncertainty robustness evaluation** (Part 16) of the degradation-aware MPC specifically, rather than only the cost-only case — extending the naive-vs-aware comparison into the MPC setting is a natural, still-modest extension beyond a pure day-ahead study.
- An **integrated EMS framework** spanning baseline → LP → degradation-aware → MPC → dashboard in one coherent, validated codebase is itself a nontrivial engineering deliverable for a final-year project, independent of any single technique's novelty.

Do not claim algorithmic novelty in the report; claim **implementation rigor, explicit trade-off quantification, and integrated scope** instead — these are true and defensible.

27 Part 26 — Implementation Roadmap

Week	Task	Priority
1	System model, parameter table, synthetic data generators (Parts 2, 3, 18)	MUST HAVE
2	Rule-based EMS + validation (Parts 5, 20)	MUST HAVE
3-4	LP formulation + linprog implementation, baseline cost results (Part 6)	MUST HAVE
5	Post-hoc degradation quantification of naive LP (Part 11)	MUST HAVE
6-7	Degradation-aware objective (quadprog), α -sweep (Part 12)	MUST HAVE
8	Comparative benchmarking table + essential scenarios (Parts 4, 13)	MUST HAVE
9	Sensitivity analysis sweeps (Part 14)	SHOULD HAVE
10-11	MPC formulation + forecast error study (Parts 15, 16)	SHOULD HAVE
12	Peak-shaving extension (Part 9)	OPTIONAL
13	Simulink cross-check for rule-based/MPC (Part 19)	OPTIONAL
14	MATLAB App Designer dashboard (Phase 6)	OPTIONAL
15	Report writing, figures, viva prep	MUST HAVE

If time becomes limited, drop in this order (last listed = drop first): Dashboard → Simulink cross-check → Peak-shaving extension → Forecast-error study depth (keep at least one moderate-error MPC run) → Sensitivity sweep breadth (keep at least battery-capacity and tariff-spread sweeps) → full MPC implementation (fall back to reporting day-ahead LP only, with MPC described as future work). **Never drop:** the rule-based baseline, the naive-LP-vs-degradation-aware comparison, and validation — these three are the project's core evidentiary chain and the whole narrative depends on them.

28 Immediate Next Steps

1. Set up the folder structure from Part 17 and commit `system_params.m` with your actual (not placeholder) numeric parameters.
2. Decide real-vs-synthetic data now (Part 18) — do not let this decision linger, since every downstream phase depends on it.
3. Implement `generate_pv_profile.m` / `generate_load_profile.m` and visually sanity-check the profiles before writing any EMS logic.
4. Implement and validate `rule_based_ems.m` against `validate_power_balance.m` — this is your first working, checkable milestone.
5. Implement `calculate_tariff.m` and construct a specific ToD tariff instance for your baseline scenario (flag its regulatory status per Part 8).
6. Implement `build_lp_problem.m` / `run_lp_ems.m` and confirm `linprog` converges (`exitflag == 1`) before trusting any output.
7. Run the Phase 2 vs. Phase 1 comparison and compute the post-hoc degradation cost (Part 11) — this is the result that motivates the rest of the project.
8. Implement the degradation-aware `quadprog` formulation (Part 12) and run the α -sweep.
9. Populate the Part 4/Phase-4 comparison table with real numbers from steps 4–8.
10. Only after steps 1–9 are solid, move to MPC (Part 15) — do not start MPC before the day-ahead LP is fully validated, since MPC reuses and depends on that code.